

- RAG Learning

3.7 Relational Databases and SQL

AQA GCSE Computer Science (8525) — Paper 2

3.7.1 Flat-File vs Relational Databases

A **database** is an organised collection of data, structured so that it can be easily stored, searched, updated and related together. Modern databases are almost always held on a computer and managed by a *database management system (DBMS)*.

Before talking about relational databases, it is easier to see the problems if we start with the simpler approach: the **flat file**. A flat-file database stores all its data in a single table — often a spreadsheet or a CSV file — with one row per item and one column per piece of data. It works well for tiny amounts of simple data, but it rapidly breaks down when the data is even slightly more complex.

A Flat-File Example

Imagine a school that keeps a single spreadsheet of every class a student is in. Each row records one enrolment — a student in a class — and copies in all the student's details and all the class's details side by side:

StudentID	StudentName	Form	ClassCode	Subject	Teacher	Room
S001	Amelia Khan	10B	CS10A	Computing	Mr Patel	G14
S001	Amelia Khan	10B	MA10A	Maths	Mrs Chen	M02
S002	Ben Okafor	10B	CS10A	Computing	Mr Patel	G14
S003	Chloe Smith	10C	CS10A	Computing	Mr Patel	G14

S003	Chloe Smith	10C	MA10A	Maths	Mrs Chen	M02
------	-------------	-----	-------	-------	----------	-----

Look at how much data is copied. Amelia's name and form appear on every row where she is enrolled. Mr Patel's name and room G14 appear on every row for CS10A. This wasted duplication is called **redundancy**, and it causes the three classic flat-file problems.

Problems With Flat Files

- **Data redundancy.** The same data is stored more than once. Each extra copy wastes storage space and, more importantly, creates opportunities for inconsistency.
- **Data inconsistency.** Because the same fact is written in many rows, it is easy for copies to disagree. One row says *Chloe Smith*, another says *Cloe Smith*; one row has room G14, another has G14a. A search for "Chloe Smith" now misses half her enrolments.
- **Update anomalies.** If Mr Patel moves from room G14 to G22, every single row that mentions him has to be changed. Miss one row and the database is broken. If you delete Ben Okafor's only enrolment row, you lose not just the enrolment but also Ben's name and form — they were only stored there.
- **Difficulty searching and reporting.** To count the number of students in 10B you must scan the whole table, because the form is repeated instead of stored once.
- **Poor data integrity.** Nothing stops someone typing `10B` on one row and `10b` on another, or entering a class code that does not exist.

The Relational Alternative

A **relational database** stores data in several *linked tables*, each holding one kind of thing. Tables are connected by **keys** rather than by copying data across, so every fact is stored in just one place.

The same school data can be split into three small tables: one for students, one for classes and one that links them (who is enrolled in what):

Student

StudentID (PK)	Name	Form
S001	Amelia Khan	10B
S002	Ben Okafor	10B
S003	Chloe Smith	10C

Class			
ClassCode (PK)	Subject	Teacher	Room
CS10A	Computing	Mr Patel	G14
MA10A	Maths	Mrs Chen	M02

Enrolment	
StudentID (FK)	ClassCode (FK)
S001	CS10A
S001	MA10A
S002	CS10A
S003	CS10A
S003	MA10A

Amelia's name now appears *once* in the whole database. Mr Patel appears *once*. To change his room you change a single cell. To find all of Chloe's classes you look up her StudentID in Enrolment, then use the class codes to look up the subject and teacher in Class. The two tables are linked by the shared values.

Benefits of a Relational Database

- **Less redundancy.** Each fact is stored once. Storage is used efficiently and there are no wasted duplicates.
- **Better data integrity and consistency.** Because a value is only stored in one place, it cannot disagree with itself. A name or room cannot be spelled two different ways in two different rows.
- **Easier updates.** Change a value once and every report or query automatically sees the new value. Update anomalies are eliminated.
- **Referential integrity.** The DBMS can stop someone entering an enrolment for a class code that does not exist, or deleting a student who still has enrolments, because it understands the foreign-key links between tables.
- **Easier searching and reporting.** A query language (SQL) can combine data from multiple tables on demand, producing any report without having to store it in advance.
- **Scalability and multi-user access.** Relational DBMSs are designed to handle thousands of concurrent users, automatic backups, recovery and security — far beyond what a shared spreadsheet can do safely.

Flat-File (redundant) vs Relational (split into linked tables)

FLAT FILE — one big table, data repeated on every row

StudentID | Name | Form | Class | Subject | Teacher | Room

S001 | Amelia Khan | 10B | CS10A | Computing | Mr Patel | G14

S001 | **Amelia Khan | 10B** | MA10A | Maths | Mrs Chen | M02 ← name/form repeated

S002 | Ben Okafor | 10B | **CS10A | Computing | Mr Patel | G14** ← class details repeated

S003 | Chloe Smith | 10C | **CS10A | Computing | Mr Patel | G14** ← repeated again!

Redundancy → inconsistency → update anomalies.



split into separate tables — each fact stored once

Student

StudentID (PK) | Name | Form

S001 | Amelia Khan | 10B

S002 | Ben Okafor | 10B

S003 | Chloe Smith | 10C

name stored once

Enrolment (link)

StudentID (FK) | ClassCode (FK)

S001 | CS10A

S001 | MA10A

S002 | CS10A

S003 | CS10A

S003 | MA10A

*only the IDs are repeated,
not the details*

Class

ClassCode (PK) | Subject | Teacher | Room

CS10A | Computing | Mr Patel | G14

MA10A | Maths | Mrs Chen | M02

class details stored once

Each fact stored in one place → no redundancy, no update anomalies.

Exam Tip: A common 4-mark question is "Give two problems with storing the data in a flat file and explain how a relational database solves them." Always name the problem (*redundancy, inconsistency, update anomaly*) and then say what the relational version does (*stores each fact once, uses keys to link tables*).

3.7.2 Relational Database Terminology

AQA expects you to know the following vocabulary precisely. The words in brackets are the formal mathematical names you sometimes meet; you can use either, but be consistent within an answer.

Term	Meaning
Database	An organised collection of related data, typically stored in one or more tables, managed by a DBMS.
Table (relation)	A two-dimensional structure that stores all the data about one kind of entity — for example, one Student table stores every student.
Record (row / tuple)	A single complete entry in a table: all the data about one student, one class, one book. One row of the table.
Field (column / attribute)	A single piece of information held about every record: <i>Name, Form, DateOfBirth</i> . One column of the table.
Primary key (PK)	A field (or combination of fields) that <i>uniquely identifies</i> each record in its table. Every table has exactly one primary key.
Foreign key (FK)	A field in one table whose value is the primary key of a different table. Foreign keys are how tables are linked to form relationships.
Composite key	A primary key made up of <i>two or more</i> fields combined, used when no single field is unique. Common in linking tables.

Primary Keys

A **primary key** is a field (or combination of fields) whose value uniquely identifies every record in a table. Two records can never share the same primary-key value, and the value can never be NULL (empty).

A good primary key is:

- **Unique** — no two records have the same value.
- **Stable** — the value does not change over time. Names and email addresses often change; IDs should not.
- **Concise** — a short value (an integer or a short code) makes comparisons and indexing fast.

Names are usually a bad primary key (two students may share a name). Instead, a school assigns every student a unique `StudentID` such as `S001`, a library gives each book a unique `BookID`, and an online shop assigns every customer a `CustomerID`. If a natural candidate key does not exist, the designer invents one — often an auto-incrementing integer (1, 2, 3, ...) called a *surrogate key*.

Foreign Keys

A **foreign key** is a field in one table whose values match primary-key values in a *different* table. It is how relational databases represent relationships between entities.

In the school example, `Enrolment.StudentID` is a foreign key that references `Student.StudentID`. The value `S001` in `Enrolment` is meaningful because it matches Amelia Khan's record in the `Student` table. A foreign key does not have to be unique in its own table — Amelia can appear many times in `Enrolment`, once for each class she takes — but every value it holds *must* exist as a primary-key value in the referenced table. That rule is called **referential integrity**.

Composite Keys

A **composite key** is a primary key made up of two or more fields combined. It is used when no single field is unique on its own. Linking tables (like `Enrolment`) almost always use a

composite key made from two foreign keys.

In Enrolment, neither StudentID nor ClassCode is unique on its own — a student is in several classes, and a class has several students — but the *combination* is unique (each student–class pair appears only once). So the primary key of Enrolment is the composite (StudentID, ClassCode). This also stops a student being accidentally enrolled in the same class twice.

A Quick Visual Summary

The Student table, with five records and three fields, illustrates all the words at once:

StudentID (<i>primary key</i>)	Name	Form
S001	Amelia Khan	10B
S002	Ben Okafor	10B
S003	Chloe Smith	10C
S004	Daniel Gray	10A
S005	Eva Novak	10C

- The whole grid is a **table**.
- Each row (e.g. S002, Ben Okafor, 10B) is one **record**.
- Each column (e.g. Name) is one **field**.
- StudentID is the **primary key**: every value is unique, non-null and does not change.
- If another table contains a column called StudentID whose values come from this one, that column is a **foreign key** to Student.

Exam Tip: Do not confuse “record” and “field” in the exam. A record is a complete row (everything about one student); a field is a single column (one piece of information about every student). A very common exam question is “State the difference between a record and a field.”

3.7.3 Relationships

A **relationship** is a link between two tables. For AQA GCSE you only need to handle the most common case, the **one-to-many** relationship, plus the classic technique of using a linking table to represent a many-to-many relationship.

One-to-Many (1:M)

In a **one-to-many** relationship, one record in table A is linked to many records in table B, but each record in B links to exactly one record in A.

Everyday examples of one-to-many relationships:

- **Customer → Orders.** One customer places many orders, but each order belongs to exactly one customer.
- **Teacher → Classes.** One teacher teaches many classes, but each class has one assigned teacher.
- **Author → Books.** (Assuming a book has one author.) One author writes many books, but each book has one author.
- **Artist → Songs.** One artist records many songs, but each song belongs to one artist.

A one-to-many relationship is implemented by adding a foreign key to the table on the “many” side. For *Customer → Orders*, the Order table has a `CustomerID` column whose values match primary keys in the Customer table:

Customer (the “one” side)		
CustomerID (PK)	Name	Email
C100	Ravi Shah	ravi@example.com
C101	Amelia Khan	amelia@example.com

Order (the “many” side)		
-------------------------	--	--

OrderID (PK)	CustomerID (FK)	OrderDate	Total
O500	C100	2026-03-04	42.90
O501	C100	2026-03-09	15.50
O502	C101	2026-03-09	9.99
O503	C100	2026-04-01	27.30

Ravi Shah (C100) has three orders; Amelia Khan (C101) has one; each order belongs to exactly one customer.

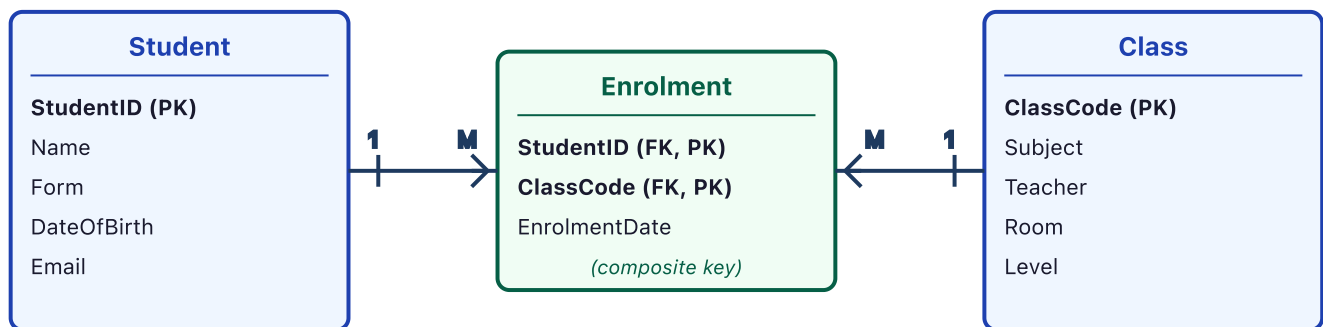
Many-to-Many (M:M) Using a Linking Table

A many-to-many relationship — many students take many classes, many customers buy many products — cannot be represented by adding a foreign key to one of the two tables. Instead, we add a **third table** (a *linking table*) that has one row for each pairing. The linking table contains two foreign keys and they together form its composite primary key.

The Student ↔ Class example from earlier is exactly this pattern. Amelia is in many classes; CS10A has many students; so Enrolment stores the pairs. Every many-to-many becomes two one-to-many relationships via the linking table.

Entity-Relationship Diagram — Student, Enrolment, Class

Many-to-many between Student and Class, implemented as two one-to-many relationships through Enrolment.



Read as: one Student has many Enrolments; one Class has many Enrolments.

→ students and classes have a many-to-many relationship via Enrolment.

Exam Tip: In an ER diagram the side marked "1" holds the primary key; the side marked "M" (or shown with a crow's foot) holds the foreign key. When you design, always ask: "one X has many Ys?" and add the FK to the Y table.

3.7.4 SQL — Querying Data With SELECT

SQL (Structured Query Language) is the standard language used to communicate with a relational database. SQL statements can *query* data (SELECT), *insert* new records (INSERT), *change* existing records (UPDATE) and *remove* records (DELETE).

SQL keywords are traditionally written in UPPERCASE and statements end with a semicolon. The DBMS is not case-sensitive about the keywords themselves but string comparisons may be case-sensitive depending on the DBMS (more on that below).

The SELECT Statement

SELECT retrieves records from one or more tables and displays them as a *result set* — another table. The basic structure is:

```
SELECT column_list
FROM table_name
WHERE condition
ORDER BY column [ASC | DESC];
```

Only `SELECT` and `FROM` are required; `WHERE` and `ORDER BY` are optional filters / sorts. All our examples use this Book table from a small library:

BookID (PK)	Title	Author	Genre	Year	Copies
B001	Holes	Louis Sachar	Fiction	1998	3
B002	The Hobbit	J.R.R. Tolkien	Fantasy	1937	5
B003	A Brief History of Time	Stephen Hawking	Science	1988	2
B004	Northern Lights	Philip Pullman	Fantasy	1995	4
B005	Matilda	Roald Dahl	Fiction	1988	6
B006	The BFG	Roald Dahl	Fiction	1982	3
B007	Stargazing	Jen Campbell	Science	2020	1

Selecting All Columns

The asterisk `*` means “every column”. This query returns every book with every field:

```
SELECT * FROM Book;
```

Result: all seven rows shown in the table above.

Selecting Specific Columns

List only the columns you want. Order them however you like:

```
SELECT Title, Author FROM Book;
```

Title	Author
Holes	Louis Sachar
The Hobbit	J.R.R. Tolkien
A Brief History of Time	Stephen Hawking
Northern Lights	Philip Pullman
Matilda	Roald Dahl
The BFG	Roald Dahl
Stargazing	Jen Campbell

Filtering With WHERE

The `WHERE` clause keeps only the rows that satisfy a condition. The comparison operators are:

Operator	Meaning	Example
<code>=</code>	Equal to	<code>Genre = 'Fiction'</code>
<code><></code>	Not equal to	<code>Genre <> 'Fiction'</code>
<code>< , ></code>	Less / greater than	<code>Year < 1990</code>
<code><= , >=</code>	Less / greater than or equal to	<code>Copies >= 3</code>
<code>AND</code>	Both conditions true	<code>Genre = 'Fiction' AND Year > 1990</code>
<code>OR</code>	Either condition true	<code>Genre = 'Fantasy' OR Genre = 'Science'</code>
<code>LIKE</code>	Pattern match (see wildcards below)	<code>Title LIKE 'The %'</code>

Strings must be enclosed in single quotes: 'Fiction'. Numbers are written without quotes.

Example 1 — simple equality

```
SELECT Title, Author
FROM   Book
WHERE  Genre = 'Fantasy';
```

Title	Author
The Hobbit	J.R.R. Tolkien
Northern Lights	Philip Pullman

Example 2 — AND combining two conditions

```
SELECT Title, Year, Copies
FROM   Book
WHERE  Genre = 'Fiction' AND Copies >= 4;
```

Title	Year	Copies
Matilda	1988	6

Example 3 — OR listing alternatives

```
SELECT Title, Genre
FROM   Book
WHERE  Genre = 'Fantasy' OR Genre = 'Science';
```

Title	Genre
The Hobbit	Fantasy
A Brief History of Time	Science

Northern Lights	Fantasy
Stargazing	Science

Pattern Matching With LIKE

The `LIKE` operator compares a text field against a *pattern*. Two wildcards are used in AQA pseudo-SQL:

- `%` matches any sequence of characters (including none).
- `_` matches exactly one character.

Pattern	Matches
'The %'	Any title starting with "The "
'%lights'	Any title ending with "lights"
'%time%'	Any title containing "time" anywhere
'_ole%'	Any title where the second, third and fourth characters are "ole"

Example 4 — LIKE with a leading wildcard

```
SELECT Title, Author
FROM   Book
WHERE  Author LIKE '%Dahl';
```

Title	Author
Matilda	Roald Dahl
The BFG	Roald Dahl

Example 5 — LIKE with wildcards on both sides

```
SELECT Title
FROM   Book
WHERE  Title LIKE '%the%';
```

Note: SQL string comparisons are usually case-insensitive in school systems, so *"The"* also matches. In AQA exam answers, treat comparisons as case-insensitive unless told otherwise.

Title
The Hobbit
Northern Lights
The BFG

Sorting With ORDER BY

Results are not guaranteed to come back in any particular order unless you ask. `ORDER BY` sorts the result set; `ASC` is ascending (A–Z, smallest first) and is the default, `DESC` is descending (Z–A, largest first).

Example 6 — alphabetical sort

```
SELECT Title, Author
FROM   Book
ORDER BY Title ASC;
```

Title	Author
A Brief History of Time	Stephen Hawking
Holes	Louis Sachar
Matilda	Roald Dahl
Northern Lights	Philip Pullman

Stargazing	Jen Campbell
The BFG	Roald Dahl
The Hobbit	J.R.R. Tolkien

Example 7 — largest number of copies first, combined with WHERE

```
SELECT Title, Copies
FROM   Book
WHERE  Genre = 'Fiction'
ORDER BY Copies DESC;
```

Title	Copies
Matilda	6
Holes	3
The BFG	3

Example 8 — sorting by two columns

```
SELECT Title, Genre, Year
FROM   Book
ORDER BY Genre ASC, Year DESC;
```

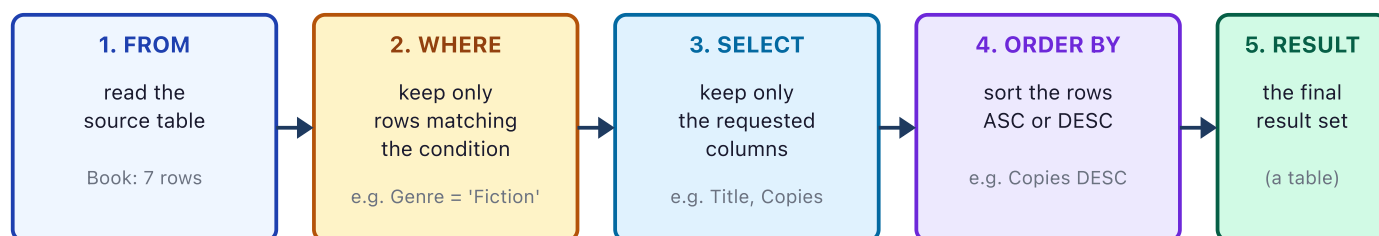
Title	Genre	Year
Northern Lights	Fantasy	1995
The Hobbit	Fantasy	1937
Holes	Fiction	1998
Matilda	Fiction	1988

The BFG	Fiction	1982
Stargazing	Science	2020
A Brief History of Time	Science	1988

(Within each genre the newest year comes first because Year is DESC; genres come in alphabetical order because Genre is ASC. Ignoring alphabetical ties is fine for AQA.)

How a SELECT query produces its result

Table(s) → WHERE filter → SELECT columns → ORDER BY sort → result set.



The DBMS may reorder these steps for efficiency, but the logical result is the same.

Exam Tip: When reading a SELECT query in an exam, work through it in the order: *FROM* (which table), *WHERE* (which rows), *SELECT* (which columns), *ORDER BY* (in what order). Candidates who rush straight to SELECT often pick the wrong columns.

3.7.5 SQL — INSERT, UPDATE, DELETE

SELECT reads; the three other commands *change* the stored data. Each has one job, and each has a form so simple that AQA expects you to know it by heart.

INSERT — adding a new record

```
INSERT INTO table_name (column1, column2, ..., columnN)
VALUES (value1, value2, ..., valueN);
```

The columns in the first list line up with the values in the second list, in the same order. Strings go in single quotes; numbers do not. Auto-incremented columns can be left out.

Example 9 — adding a new book

```
INSERT INTO Book (BookID, Title, Author, Genre, Year, Copies)
VALUES          ('B008', 'Coraline', 'Neil Gaiman', 'Fantasy', 2002, 2);
```

After running this, the Book table has an eighth row:

BookID	Title	Author	Genre	Year	Copies
B008	Coraline	Neil Gaiman	Fantasy	2002	2

If you list every column in the table's natural order, you can also write the shorter form `INSERT INTO Book VALUES ('B008', 'Coraline', 'Neil Gaiman', 'Fantasy', 2002, 2);`. Naming the columns is safer because it still works if the table's structure changes.

UPDATE — changing existing records

```
UPDATE table_name
SET     column1 = newvalue1, column2 = newvalue2, ...
WHERE  condition;
```

The `SET` clause lists the new values; the `WHERE` clause specifies which rows to change. **Always use a WHERE clause** — forgetting it updates every row in the table.

Example 10 — adding a copy of Holes

```
UPDATE Book
SET     Copies = Copies + 1
WHERE  BookID = 'B001';
```

The Copies value for B001 changes from 3 to 4; all other rows are untouched.

Example 11 — changing multiple fields in one row

```
UPDATE Book
SET     Genre = 'Science Fiction', Copies = 3
WHERE   Title = 'The Hobbit';
```

Result: the Genre of The Hobbit becomes *Science Fiction* and the number of Copies becomes 3 (up from 5).

Example 12 — changing many rows at once

```
UPDATE Book
SET     Genre = 'Children''s Fiction'
WHERE   Author = 'Roald Dahl';
```

Both Dahl books (Matilda and The BFG) have their Genre changed. Notice that the apostrophe in “Children's” is escaped by doubling it (`''`). This is how SQL handles a single quote inside a string.

DELETE — removing records

```
DELETE FROM table_name
WHERE      condition;
```

Deletes every row that matches the condition. Again, **forgetting the WHERE clause deletes every row in the table.**

Example 13 — deleting a specific book

```
DELETE FROM Book
WHERE      BookID = 'B007';
```

The row for “Stargazing” is removed. Other rows are untouched.

Example 14 — deleting all books with no copies

```
DELETE FROM Book
WHERE      Copies = 0;
```

If any books have Copies = 0 they are all removed. If there are none, the query runs but changes nothing.

Exam Tip: In a 2 or 3-mark question you will often be asked to write ONE SQL statement. Read the question carefully: "add" = INSERT, "change" / "increase" / "amend" = UPDATE, "remove" = DELETE, "find" / "list" / "display" = SELECT. Also: *a/ways* include the WHERE clause for UPDATE and DELETE unless the question says "every".

3.7.6 Joining Tables

Because relational databases split data across tables, most useful queries need to combine data from two tables at once. AQA does not require the `JOIN` keyword; it expects you to use the simpler "two tables in the FROM clause" syntax with a WHERE clause that matches the foreign key to the primary key.

The Basic Two-Table Pattern

```
SELECT columns_from_either_table
FROM tableA, tableB
WHERE tableA.primary_key = tableB.foreign_key
      AND other_conditions;
```

If you do not include the linking condition (`tableA.pk = tableB.fk`), the DBMS returns every combination of rows from the two tables — which is almost always nonsense.

Example Data — Customer and Order

Customer		
CustomerID (PK)	Name	Email
C100	Ravi Shah	ravi@example.com
C101	Amelia Khan	amelia@example.com

C102	Ben Okafor	ben@example.com
------	------------	-----------------

`Order`			
OrderID (PK)	CustomerID (FK)	OrderDate	Total
O500	C100	2026-03-04	42.90
O501	C100	2026-03-09	15.50
O502	C101	2026-03-09	9.99
O503	C100	2026-04-01	27.30
O504	C102	2026-04-05	62.00

Example 15 — list every order with the customer's name

```
SELECT  Customer.Name, `Order`.OrderID, `Order`.Total
FROM    Customer, `Order`
WHERE   Customer.CustomerID = `Order`.CustomerID
ORDER BY Customer.Name ASC;
```

Name	OrderID	Total
Amelia Khan	O502	9.99
Ben Okafor	O504	62.00
Ravi Shah	O500	42.90
Ravi Shah	O501	15.50
Ravi Shah	O503	27.30

When a column name exists in more than one table (here *CustomerID*) you must write `tablename.columnname` . If the column is unique you can just write the column name on its own,

but fully qualifying is safer.

Example 16 — filtering the joined result

Find every order placed by Ravi Shah, showing only the date and total:

```
SELECT  OrderDate, Total
FROM    Customer, `Order`
WHERE   Customer.CustomerID = `Order`.CustomerID
      AND Customer.Name = 'Ravi Shah';
```

OrderDate	Total
2026-03-04	42.90
2026-03-09	15.50
2026-04-01	27.30

Example 17 — three tables

The same idea scales to three tables. Combining our school Student, Class and Enrolment:

```
SELECT  Student.Name, Class.Subject, Class.Teacher
FROM    Student, Enrolment, Class
WHERE   Student.StudentID = Enrolment.StudentID
      AND Enrolment.ClassCode = Class.ClassCode
      AND Student.Form      = '10B'
ORDER BY Student.Name, Class.Subject;
```

This lists every subject that every 10B student is taking, along with who teaches it. With n tables you need $n - 1$ linking conditions.

Exam Tip: A very common AQA mark-scheme check-list for a join question is: (1) correct tables in `FROM` (1 mark), (2) correct linking condition in `WHERE` (1 mark), (3) correct filter condition (1 mark), (4) correct columns in `SELECT` (1 mark). Missing the linking condition is the single most common reason candidates lose marks.

3.7.7 Designing a Database

Database design is one of the few topics where the AQA exam routinely asks you to *produce* something, not just describe it. You will be given a scenario and asked to identify entities, attributes and keys, or to draw a simple schema. The process below always works.

Step 1: Identify the Entities

An **entity** is a real-world thing about which we store data — usually a noun in the scenario description. Each entity becomes a table. Pick nouns that have their own set of facts, not attributes of other things.

For a scenario “A vet's surgery treats pets. Each pet is owned by an owner, and each visit to the surgery is recorded” the entities are:

- **Owner** (a person who owns pets)
- **Pet** (an animal registered at the surgery)
- **Visit** (one appointment)

Step 2: Identify the Attributes

An **attribute** is a single fact about an entity — the fields in the table. Look for the information the scenario mentions, and add obvious extras:

- Owner: OwnerID, Name, Address, Phone.
- Pet: PetID, Name, Species, Breed, DateOfBirth, OwnerID (FK).
- Visit: VisitID, Date, PetID (FK), Reason, Vet.

Step 3: Pick the Primary Keys

For each table, choose a field that uniquely identifies a record and never changes. If no natural key exists, invent an ID.

- **Owner.OwnerID** — unique integer or code.
- **Pet.PetID** — unique; Name is a bad choice because two pets can be called Lucky.
- **Visit.VisitID** — unique; Date + PetID might also work but an ID is simpler.

Step 4: Work Out the Relationships and Add Foreign Keys

For each pair of entities ask “one X has many Ys, or the other way round?”. Put the foreign key on the “many” side:

- One Owner has many Pets → add `OwnerID` to Pet.
- One Pet has many Visits → add `PetID` to Visit.

If the relationship is many-to-many, add a linking table (as in the school example).

Step 5: Draw the Schema

A *database schema* is a diagram showing every table, every field, the primary key of each table, and the foreign keys that link tables together. A simple text-style schema looks like this:

```
Owner ( OwnerID, Name, Address, Phone )
Pet   ( PetID,   Name, Species, Breed, DateOfBirth, *OwnerID )
Visit ( VisitID, Date, *PetID, Reason, Vet )
```

(Underlining marks the primary key; a leading asterisk marks a foreign key. AQA accepts any clear convention as long as you explain it.)

Or drawn as entity boxes linked with 1:M lines:

Owner <u>OwnerID</u> Name Address Phone	1 → M	Pet <u>PetID</u> Name Species Breed DateOfBirth *OwnerID	1 → M	Visit <u>VisitID</u> Date *PetID Reason Vet
--	-------	---	-------	---

Checklist for Exam Schemas

- Every table has exactly one primary key (or a clearly indicated composite key).
- Every one-to-many relationship has a foreign key on the “many” side, matching the type and name of the primary key it references.

- No attribute is repeated across tables except foreign keys.
- Attributes belong in the table they are *about*: the owner's name goes in Owner, not Pet.
- Many-to-many relationships have a linking table with a composite primary key of the two foreign keys.

Exam Tip: When you are asked to “draw a database schema” for 4–6 marks, always show: (1) at least two tables, (2) the primary key of each (underlined or marked PK), (3) at least one foreign key, (4) a line linking the PK and FK. Missing the link line is the most common reason students drop a mark.

3.7.8 Applying to Scenarios

The last section pulls every skill together on small realistic scenarios. Every GCSE 8525 paper includes at least one multi-part database question of this form: “here is a scenario, here are some tables, write the SQL to ...”. The questions nearly always use a library, a school or an online shop.

Scenario A — A Small Library

The library keeps two tables:

Book					
BookID (PK)	Title	Author	Genre	Year	Copies
B001	Holes	Louis Sachar	Fiction	1998	3
B002	The Hobbit	J.R.R. Tolkien	Fantasy	1937	5
B003	A Brief History of Time	Stephen Hawking	Science	1988	2
B004	Matilda	Roald Dahl	Fiction	1988	6

Loan					
------	--	--	--	--	--

LoanID (PK)	BookID (FK)	MemberName	DateOut
L01	B002	Priya Patel	2026-04-10
L02	B004	Ben Okafor	2026-04-12
L03	B001	Priya Patel	2026-04-14

Task A1: list the titles of every Fiction book, alphabetically.

```
SELECT Title
FROM Book
WHERE Genre = 'Fiction'
ORDER BY Title ASC;
```

Result: Holes, Matilda.

Task A2: list every book published before 1990 with 3 or more copies.

```
SELECT Title, Year, Copies
FROM Book
WHERE Year < 1990 AND Copies >= 3;
```

Result: The Hobbit (1937, 5 copies); Matilda (1988, 6 copies).

Task A3: add a new book.

```
INSERT INTO Book (BookID, Title, Author, Genre, Year, Copies)
VALUES ('B005', 'Northern Lights', 'Philip Pullman', 'Fantasy', 1995, 4);
```

Task A4: a book has been damaged; reduce its Copies by one.

```
UPDATE Book
SET Copies = Copies - 1
WHERE BookID = 'B002';
```

Task A5: a book has been written off entirely; remove it.

```
DELETE FROM Book
WHERE      BookID = 'B003';
```

Task A6: list the titles of every book currently on loan, with the member's name.

```
SELECT Book.Title, Loan.MemberName, Loan.DateOut
FROM    Book, Loan
WHERE   Book.BookID = Loan.BookID
ORDER BY Loan.DateOut ASC;
```

Result: The Hobbit / Priya Patel / 2026-04-10; Matilda / Ben Okafor / 2026-04-12; Holes / Priya Patel / 2026-04-14.

Scenario B — An Online Shop

The shop has four tables: Customer, Product, `Order`, and OrderLine (because each order can contain several products — a many-to-many between `Order` and Product):

```
Customer ( CustomerID, Name, Email, Postcode )
Product  ( ProductID, Name, Category, Price, Stock )
`Order`  ( OrderID,      *CustomerID, OrderDate )
OrderLine ( *OrderID, *ProductID, Quantity )
```

OrderLine has a composite primary key (OrderID, ProductID) : each product appears at most once per order.

Task B1: list every product in the category "Games" priced £20 or less.

```
SELECT Name, Price
FROM    Product
WHERE   Category = 'Games' AND Price <= 20
ORDER BY Price ASC;
```

Task B2: add a new customer.

```
INSERT INTO Customer (CustomerID, Name, Email, Postcode)
VALUES              ('C250', 'Grace Thomas', 'grace@example.com', 'AB1 2CD');
```

Task B3: a product's price is being reduced by 10%.

```
UPDATE Product
SET    Price = Price * 0.9
WHERE  ProductID = 'P404';
```

Task B4: list the names of all customers who have placed an order.

```
SELECT Customer.Name
FROM    Customer, `Order`
WHERE   Customer.CustomerID = `Order`.CustomerID
ORDER BY Customer.Name ASC;
```

(This may list the same name more than once if a customer has several orders. Removing duplicates uses `SELECT DISTINCT`, which is beyond the GCSE spec.)

Task B5: remove a customer's account.

```
DELETE FROM Customer
WHERE     CustomerID = 'C250';
```

In a real DBMS, referential integrity may prevent this delete if the customer still has orders — the DBMS will reject the DELETE rather than leave orphaned order rows.

Scenario C — A School Music Service

One last practice scenario to pull everything together. A music service records which *instruments* each *pupil* is learning, and which *teacher* teaches which instrument lesson each week.

Entities: **Pupil**, **Instrument**, **Teacher**, **Lesson**. Relationships: each pupil has many lessons, each teacher gives many lessons, each lesson is for one instrument.

```
Pupil      ( PupilID,      Name, YearGroup )
Instrument ( InstrumentID, Name )
Teacher    ( TeacherID,    Name )
Lesson     ( LessonID,      *PupilID, *InstrumentID, *TeacherID, DayOfWeek, StartTime )
```

Lesson is a *single* table that joins three others through three foreign keys. A lesson belongs to one pupil, one instrument and one teacher — three one-to-many relationships, so three foreign keys but a single primary key (LessonID).

Task C1: list every lesson on Monday, with the pupil's name and the instrument.

```
SELECT  Pupil.Name, Instrument.Name, Lesson.StartTime
FROM    Pupil, Instrument, Lesson
WHERE   Pupil.PupilID      = Lesson.PupilID
        AND Instrument.InstrumentID = Lesson.InstrumentID
        AND Lesson.DayOfWeek = 'Monday'
ORDER BY Lesson.StartTime ASC;
```

Task C2: move every Tuesday piano lesson to Wednesday.

```
UPDATE Lesson
SET    DayOfWeek = 'Wednesday'
WHERE  DayOfWeek = 'Tuesday'
        AND InstrumentID = (SELECT InstrumentID FROM Instrument WHERE Name = 'Piano');
```

(The subquery is beyond the spec but included here to show how the same task is done in practice; a GCSE-level answer would assume the piano's InstrumentID is known, e.g. `AND InstrumentID = 'I03'`.)

Putting It All Together — The Exam Approach

1. Read the scenario and **underline the nouns** (potential entities) and the facts about them (attributes).
2. For each entity, pick or invent a **primary key**.
3. Work out **relationships**: “one X has many Ys”. Add foreign keys on the many side.
4. If a many-to-many exists, add a **linking table** with a composite primary key.
5. Draw the schema — boxes with PK underlined and FK starred, lines between PK and FK pairs.
6. For SQL questions, write the keyword first, check quotes around strings, include the **WHERE** clause for UPDATE/DELETE, and for joins always include the *linking condition* alongside any other filters.

7. Finally, for evaluate questions comparing flat files and relational databases, always name at least two problems the flat file has (*redundancy, inconsistency, update anomaly*) and at least two benefits the relational database delivers (*each fact stored once, referential integrity, easier updates, easier querying*).

Exam Tip: If you remember just one thing from this topic, make it this: *a relational database splits data into tables so that every fact is stored in one place, and uses primary keys and foreign keys to link tables*. That one sentence, well placed, unlocks the majority of the marks in any 3.7 question.